

# 동작 원리를 끝까지 이해하고, 이해한 것을 다시 사용할 수 있도록 남기는 엔지니어입니다.

명확히 설명되어 있지 않은 동작도 코드와 실험, 비교 검증을 통해 파악하고, 필요한 기준과 구조를 만들어갑니다.

파악한 내용을 저만의 지식으로 두지 않습니다. 개인의 경험에 의존하던 빌드 규칙을 모듈과 가이드로 옮기고 전사 세미나로 공유했습니다.

필요한 것은 직접 만들고 운영합니다. 가족이 쓰는 서비스를 홈 서버에서 운영하며 시스템을 만드는 것뿐 아니라 안정적으로 유지하는 방법을 배우고 있습니다.

EMAIL canon1107@naver.com

PHONE 010.2089.6967

## Experience

티맥스 티베로 데이터베이스 엔진 연구원

2024.08 - 현재

Optimizer 기능 개발과 이슈 분석, DBMS 기능 구현을 수행했고, Jam 기반 빌드 시스템을 CMake로 전환했습니다.  
현재는 대규모 C 코드베이스의 코드 품질을 높이기 위한 연구를 진행하고 있습니다.

주식회사 일상도시 도시재생 분야 연구

2020.08 - 2023.05

동인천역 주변 역사문화자산 발굴 및 아카이빙, 도시재생활성화계획 수립 용역을 수행했습니다.

## Education

삼성 청년 SW 아카데미 10기

2023.06 - 2024.05

1학기 성적우수상 · 1학기 관통 프로젝트 최우수상 · 2학기 특화 프로젝트 우수상

성균관대학교 건축학과

2016.03 - 2022.02

학사 졸업

## Certificates

정보처리기사

2024.09 취득

SQL 개발자

2023.12 취득

## Skills

Language · DBMS

C

SQL

Tibero

Oracle

Build · Infra

CMake

Linux

Docker

Shell

# Projects

티맥스 티베로 2025.08 - 2026.04

## DBMS 빌드 시스템 Jam → CMake 전환

분석 설계 구현

상용 DBMS의 빌드 시스템을 레거시 도구 Jam에서 CMake로 전환한 프로젝트입니다.

### 배경

- 장기간 유지된 Jam 기반 빌드 시스템은 빌드 규칙과 예외가 코드와 스크립트에 흩어져 있어 파악과 수정이 어려웠습니다.
- 빌드 파일 간 의존성 누락으로 병렬 빌드 수준을 높이는 데 한계가 있었고, 개발 중 빌드 실패로 재시도가 필요한 경우가 발생했습니다.

### 작업 내용

- 문서화되지 않은 Jam 빌드 규칙을 소스코드와 기존 빌드 흐름을 기반으로 역분석했습니다.
- 분석 결과를 바탕으로 CMake 빌드 시스템을 설계·구현하고, 의존성을 명시적으로 선언하는 구조로 재편했습니다.
- Linux / Windows / AIX / Solaris 플랫폼별 빌드·링크 이슈를 대응했습니다.
- 빌드 파일 작성 방식과 사용법을 가이드 문서로 정리하고 전사 세미나로 공유했습니다.
- 전환 후 연구원들의 문의와 개선 요청에 대응하며 빌드 시스템을 안정화했습니다.

### 성과

- 동일 형상 기준 빌드 시간이 기존 Jam 약 34~38분에서 CMake 전환 후 약 24~26분 수준으로 단축되어, 약 30% 개선 효과를 확인했습니다.
- 의존성 누락으로 인한 반복 빌드 실패를 개선하고, 재시도 없이 빌드되는 환경을 확보했습니다.
- 메인 개발 브랜치 적용 후 다른 버전 브랜치에서도 전환 요청을 받아 확대 적용했습니다.
- 빌드 규칙과 작성 방식을 문서화해, 신규 작성자가 가이드를 기반으로 빌드 파일을 작성할 수 있는 환경을 마련했습니다.

## Optimizer Pending Statistics 기능 개발

스펙 분석

설계

구현

통계 수집 시점과 실행계획 반영 시점을 분리하는 Oracle Pending Statistics 기능을 Tibero에 신규 개발했습니다.

### 배경

- 운영 DB에서는 통계 수집 시 실행계획이 바뀔 수 있어, 통계 갱신 자체가 성능 변동 리스크가 될 수 있었습니다.
- 통계를 미리 수집해 검증한 뒤 반영할 수 있는 Oracle의 Pending Statistics 기능을 고객사가 요청했습니다.

### 작업 내용

- Oracle 공식 문서로 기능 스펙을 파악하고, 관련 view 조회와 통계 반영 실험을 통해 실제 저장 구조와 동작 흐름을 역추적했습니다.
- Pending Statistics의 수집·저장·조회·반영 로직을 Tibero 엔진 구조에 맞게 설계·구현했습니다.
- 실행계획 생성 과정에서 Pending Statistics 사용 여부를 제어하도록 Optimizer의 통계 로딩 경로와 파라미터 처리 흐름을 수정했습니다.
- 테이블/인덱스/컬럼/파티션 단위 테스트 시나리오를 구성하고, Oracle과 실행계획을 비교 검증했습니다.

### 성과

- 통계 수집 시 실행계획이 변경되는 리스크를 줄일 수 있도록 Pending Statistics 기능을 구현했고, **Tibero 7.2.4 릴리즈에 포함**했습니다.
- 테이블/인덱스/컬럼/파티션 단위 주요 시나리오에서 Oracle과 실행계획을 비교 검증했습니다.
- Pending Statistics가 없을 때 Optimizer가 Dynamic Sampling으로 동작하는 이슈를 발견하고, 기존 통계를 사용하도록 설계를 수정했습니다.

## Optimizer 모듈 이슈 분석 및 대응

이슈 재현

원인 분석

수정

내부 테스트 및 고객사 환경에서 발생한 Optimizer 관련 이슈를 재현하고 원인을 분석해 대응했습니다.

### 주요 사례

- DB 버전 업그레이드 후 특정 SQL이 느려진 이슈를 분석했습니다. 버전별 SQL Trace와 실행계획을 비교해 Index Scan이 Full Scan으로 바뀐 지점을 확인하고, 통계가 실행계획에 반영되지 않던 원인을 수정했습니다.
- 특정 집계 쿼리에서 서버가 크래시나는 이슈를 최소 재현 케이스로 줄였습니다. 정상/비정상 케이스를 비교해 여러 인자를 받는 집계 함수가 특정 실행 경로에서 잘못 처리되는 문제를 수정했습니다.
- 병렬 쿼리 결과가 실제 데이터와 다르게 반환되는 이슈를 분석했습니다. 병렬/비병렬 실행계획을 비교해 병렬 계획 생성 과정의 데이터 분배 문제를 찾아 수정했습니다.

### 성과

- SQL이 실행계획으로 만들어지는 과정을 코드 경로를 따라 분석하며, **통계와 파라미터에 따라 실행계획이 달라지는 구조**를 이해했습니다.
- 성능 저하·크래시·결과 오류처럼 유형이 다른 **Optimizer 이슈를 최소 재현부터 수정까지 담당**했습니다.
- 최소 재현, 실행계획 비교, 영향 범위 확인을 이슈 분석 절차로 정리해 유사 이슈 대응에 활용했습니다.

## DBMS C 코드베이스 의존성 정리 및 소스 품질 개선

소스 품질 개선

DBMS C 코드베이스의 include 의존성을 정리하고, 모듈별 독립 빌드가 가능한 구조로 개선하고 있습니다.

### 배경

- 장기간 유지된 C 코드베이스에서 헤더와 모듈 간 의존성이 누적되어, 실제 필요한 의존성과 전이 의존에 기대는 구조를 구분하기 어려웠습니다.
- 모듈 간 의존이 강하게 얹혀 있어 특정 모듈만 독립적으로 빌드하거나 검증하기 어려운 구조였습니다.

### 작업 내용

- 유사한 C/C++ 코드베이스 정리 사례와 도구 적용 범위를 조사하고, 프로젝트 특성에 맞는 단계적 정리 방향을 세웠습니다.
- include 사용 관계를 분석해 불필요한 include 제거, 전방 선언 대체, 더 작은 provider header 교체 기준을 정리했습니다.
- 빌드 산출물의 symbol 정보를 기반으로 모듈별 제공/사용 관계를 추적해 모듈 간 의존성을 분석했습니다.

### 성과

- 불필요한 include와 전이 의존을 정리한 결과, 공통 TU 기준 전처리 라인 수 4.54%, byte size 5.13%, **clean build 시간 평균 6.4% 감소**를 확인했습니다.
- include 정리를 사전 작업으로 진행한 뒤, 모듈 간 직접 참조와 순환 의존을 줄이는 모듈화 작업으로 확장하고 있습니다.

## 홈 서버 구축 및 운영

구축

운영

개선

개인과 가족이 사용하는 사진 백업·문서 관리·Git 서비스를 직접 구축하고, 운영하는 홈 서버 인프라입니다.

### 배경

- 가족 사진을 한곳에 모아 안전하게 보관하고, 외부에서도 접근할 수 있는 백업·조회 환경이 필요해 시작했습니다.
- 개인 개발, 기술 검증, 서비스 운영 테스트를 위한 독립 서버 환경으로도 활용하고 있습니다.

### 작업 내용

- 방화벽, 리버스 프록시, 도메인 설정을 구성해 외부 접근 경로를 단일화하고, 서비스별 접근 정책을 문서화해 관리했습니다.
- 사진 백업, 문서 관리, Git 등 필요한 서비스를 Docker 기반으로 배포하고 사용 목적에 따라 추가·삭제하며 운영했습니다.
- iCloud → NAS → 외장하드로 이어지는 백업 흐름을 구성해 사진 데이터 손실 위험을 줄였습니다.
- 서버와 컨테이너 상태, 로그, 디스크 사용량을 모니터링해 장애 징후를 확인할 수 있도록 구성했습니다.

### 운영 중 개선

- 디스크 용량 부족으로 인증 서비스가 중단되고 다른 서비스 접근 장애로 이어진 경험을 바탕으로, 디스크 사용량 모니터링을 추가했습니다.
- 테스트용 VM을 분리해 새로운 서비스나 설정을 검증하더라도 실제 사용 중인 서비스에 영향을 주지 않도록 운영 환경과 실험 환경을 분리했습니다.

SSAFY

2024.04 - 2024.05

## Omegi : 분산 시스템 오류 추적 서비스

Python 자동 계측 라이브러리 개발

로그 수집·가공 파이프라인

GitHub ↗

MSA 환경에서 여러 서비스에 걸친 오류의 흐름을 추적하고 기록하는 서비스입니다.

### 수행 내용

- OpenTelemetry 기반 함수 단위 자동 계측 라이브러리를 개발했습니다.
- Decorator로 Span을 생성해 함수 호출 간 부모-자식 관계를 유지하고, Kafka로 trace를 전송하는 Exporter를 구현했습니다.
- Java·Python 서비스 간 trace가 이어지지 않는 문제를 W3C Trace Context로 전파 형식을 통일해 해결했습니다.
- Kafka → Redis 작업 큐 → Elasticsearch 흐름으로 순서 없이 도착하는 로그를 하나의 trace로 모아 가공했습니다.

SSAFY

2024.02 - 2024.04

## Englising : 팝송 기반 영어 학습 게임

데이터 수집 파이프라인

게임 API

배포 환경 구성

GitHub ↗

팝송 가사 빈칸 맞추기로 영어를 학습하는 웹 서비스입니다.

### 수행 내용

- Spotify API, 가사, YouTube 데이터를 수집해 게임 출제에 필요한 형태로 가공했습니다.
- 수집 파이프라인을 Queue와 Worker 기반 구조로 분리해 단계별 수정과 재시작이 가능하도록 개선했습니다.
- WebSocket 기반 멀티플레이 진행 로직과 Redis 기반 게임 상태 저장 기능을 구현했습니다.
- Docker, Jenkins, Nginx 기반 자동 배포 환경을 구성했습니다.

SSAFY

2024.01 - 2024.02

## Togeduck : 생일카페 지도 Android 앱

팀장

Kotlin 화면 개발

WebSocket 실시간 위치 공유·채팅 기능

GitHub ↗

생일카페 정보를 지도로 탐색하고 실시간으로 소통하는 Android 앱입니다.

### 수행 내용

- Kotlin 기반 Android 주요 화면을 개발했습니다.
- 지도 위 생일카페 마커 표시와 클러스터링 기능을 구현했습니다.
- WebSocket 기반 사용자 실시간 위치 공유와 채팅 기능을 구현했습니다.
- 사용자별 위치 좌표를 관리해 잦은 마커 갱신 중에도 실시간 위치 표시가 안정적으로 동작하도록 개선했습니다.